

P2964R4

User-defined element types in `std::simd` through trait-based vectorizable definition

Published Proposal, 2026-05-12

This version:

<http://wg21.link/P2964R4>

Authors:

[Daniel Towner](#) (Intel)

[Ruslan Arutyunyan](#) (Intel)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper proposes extending `std::simd` to support user-defined element types by extending the closed list of vectorizable types with a trait-based definition. This minimal change enables type safety, strong typedefs, enumerations, and `std::byte` while maintaining full backward compatibility.

Table of Contents

1	Revision History
1.1	R3 → R4
1.2	R2 → R3
1.3	R1 → R2
1.4	R0 → R1
2	Introduction
2.1	Evolution and Design Foundation
2.2	What This Proposal Enables
2.3	Core Proposal
2.4	Scope and Future Directions
2.4.1	In Scope: Element-wise Semantics
2.4.2	Deliberately Out of Scope: Heterogeneous Operations
2.4.3	Forward Compatibility
2.4.4	Future Extension: Heterogeneous Operations
3	Motivation
3.1	Type Safety and Strong Typedefs
3.2	Signal and Media Processing Types
3.3	Enumerations
3.4	<code>std::byte</code>
3.5	Compound Types
4	Understanding Type Constraints
4.1	Trivially Copyable Constraint

4.2	Size Constraint
4.3	Padding and Bit Representation
4.4	Opt-Out Mechanism
4.5	Banned Standard Library Types
4.6	std::byte as a built-in vectorizable type
4.7	Summary of Constraints
5	Operations on User-Defined Types
5.1	Operator Constraints
5.2	Conversions and Casts
5.2.1	Value-Preserving Conversions
5.3	Maths Functions
5.4	Reductions
5.4.1	reduce_min and reduce_max
5.5	Load and Store Operations
5.6	Copy Operations
5.7	Implementation Considerations
5.8	ABI Selection for User-Defined Types
6	Customization Points
6.1	Operation Customization
6.2	Conversion Customization
7	Implementation Experience
7.1	Test Implementation
7.2	Successful Inference Cases
7.3	Identified Limitation
7.4	Implications for Customization
7.5	Implementation Impact
8	Extended Enum and Byte Support
9	Proposed Wording
9.1	Modify [simd.syn]
9.2	Modify [simd.general]
9.3	Add [simd.convert.tag]
9.3.1	Conversion tag types [simd.convert.tag]
9.4	Add [simd.disable] after [simd.general]
9.4.1	Disabling vectorization [simd.disable]
9.5	Add exposition-only concepts to [simd.expos]
9.6	Add [simd.cust] — Customization points
9.6.1	Customization points [simd.cust]
9.6.1.1	Operation customization [simd.cust.op]
9.6.1.2	Conversion customization [simd.cust.convert]
9.7	Modify [simd.ctor] broadcasting constructor
9.8	Modify [simd.ctor] converting constructor
9.9	Modify [simd.unary]
9.10	Modify [simd.binary]
9.11	Modify [simd.cassign]
9.12	Modify [simd.comparison]
9.13	Add overload for to_underlying
9.14	Add overload for to_integer
9.15	Feature test macro [version.syn]

10 Conclusion

11 Acknowledgements

12 Appendix: Customization Point Technical Details

12.1 Dual Dispatch Strategy

12.2 Complete Example with Selective Customization

13 Appendix: Assembly Code Examples

References

Informative References

1. Revision History

1.1. R3 → R4

Updated after SG6 review at Croydon:

- Made customization points parts of the proposal, rather than optional extensions.
- Retained closed-list approach for built-in vectorizable types rather than replacing it with trait-based definition, to preserve semantic distinction and avoid unintended consequences.
- Explored how to customise `min/max` reductions.
- Removed `alignof` constraint
- Stronger reasoning for banning pointers.
- Clarified architectural boundary for heterogeneous operations, noting that support for quantities and units library should be deferred.
- Made `to_underlying` and `to_integer` part of the proposal rather than optional extensions.
- Noted reason for choosing fixed element sizes.

1.2. R2 → R3

- Fix wording issues
- Fix rendering issues

1.3. R1 → R2

- Changed approach from customization-focused to trait-based constraints
- Moved customization points to design alternative section
- Provide many implementation examples

1.4. R0 → R1

- Incorporated SG1 and SG6 feedback from 2024 Tokyo meeting
- Added restrictions on element types
- Added inferencing as valid method for constructing simd operators
- Changed from opt-in to opt-out mechanism

§ 2. Introduction

The C++ standard library includes data-parallel types in the `<simd>` header, currently restricting element types to a closed list of *built-in vectorizable* types: arithmetic types and `std::complex` specializations. This paper proposes a minimal change to the specification in which this list is extended. Firstly, `std::byte` is added to the closed list as a standard library type with fixed semantics that the implementation handles directly. Secondly, trait-based constraints define a second category of *user-defined vectorizable* types as any type that is trivially copyable, of appropriate size, and not explicitly opted out. All existing built-in vectorizable types retain their current semantics unchanged, while support naturally extends to enumerations and user-defined types.

Although the change is fairly minimal, this paper thoroughly explores the implications of the changes, including detailed design of type constraints, operator semantics, conversions, and implementation experience. This comprehensive approach is in response to committee feedback requesting evidence that the approach works in practice and careful consideration of edge cases, particularly around type conversions and compiler optimization capabilities.

§ 2.1. Evolution and Design Foundation

Earlier revisions of this proposal focused on providing explicit customization mechanisms for user-defined types. Committee feedback encouraged us to explore element-wise inference instead, making use of the Working Draft's wording in which everything is defined in terms of element operations and element-wise application of those operations. This led to a key question: can modern compilers effectively auto-vectorize element-wise operations on user-defined types? Our investigation showed that leading optimizing compilers can indeed do this remarkably well, and this observation became the foundation of our design.

By relying on compiler optimization, we can open `simd` to user-defined types without requiring customization points for basic operations. This meant we could achieve the desired functionality by simply changing which types are allowed to be elements (i.e., what a vectorizable type is), without modifying operation semantics. The elegance of this approach is that changing only the gate-keeper logic provides the extension we need to support not only user-defined types, but other useful types like enumerations.

During the last committee meeting, concerns were raised about the performance implications of this approach - what if compilers failed to vectorize the code? To address these concerns we implemented our proposal in Intel's `std::simd` implementation and tested it across multiple generations of Intel architectures with various user-defined types, enumerations, strong typedefs, and specialized DSP types (saturating arithmetic and fixed-point). Implementation experience (§7 [Implementation Experience](#)) demonstrates that with current leading compilers (Clang and Intel oneAPI), these types can generate assembly identical to built-in arithmetic types for standard operations. This proves the approach is viable. Compilers that don't yet optimize as well will improve over time.

§ 2.2. What This Proposal Enables

This proposal allows `simd` to support user-defined types, `std::byte`, enumerations, and other types beyond the current closed list of types. The key requirement is that element-wise application of the scalar operations makes sense for the type.

Examples of types that become vectorizable:

- **User-defined types for type safety:** `struct Meters { float value; };` - strong typedefs that wrap primitives
- **Enumerations:** `enum class Color : uint32_t { Red, Green, Blue };` - type-safe alternatives to raw integers
- **`std::byte`:** for packet processing and binary data manipulation
- **Specialized arithmetic types:** saturating integers, fixed-point numbers with custom operators
- **Simple aggregates:** `struct RGBA { uint8_t r, g, b, a; };` - small value types with element-wise semantics

What these types share is that the desired SIMD behavior is straightforward: if a type `T` has `operator+`, then `simd<T>` should provide element-wise `operator+` with the same semantics. The scalar operations on `T` define what `simd<T>` should do.

We did find that while element-wise inference works well for most operations (arithmetic, comparisons, permutations, broadcasts), it can occasionally struggle with complex algorithms like reductions or user-defined operators containing

branching. To address this, we propose an ADL-based customization mechanism (`simd_operator` for operations, `simd_convert` for conversions) that allows users to provide optimized implementations for specific operations while maintaining element-wise inference as the default. This hybrid approach provides a solid foundation that works well in practice while enabling targeted optimization when necessary.

This proposal does NOT address *heterogeneous type operations* where operands have different types and produce a third type (e.g., dimensional analysis where `Length / Time -> Speed`). Such operations represent a fundamentally different design space requiring type-level computation and are explicitly out of scope.

2.3. Core Proposal

The core idea of our proposal is to extend the set of vectorizable types by adding a second category of user-defined vectorizable types alongside the existing built-in vectorizable types. A type `T` is a user-defined vectorizable type if:

- `std::is_trivially_copyable_v<T>` is true
- `sizeof(T)` is 1, 2, 4, 8, or 16
- `std::disable_vectorization<T>` is false

The built-in vectorizable list is retained rather than replaced because it carries meaning throughout the working draft. Retaining it also makes visible the semantic distinction between built-in vectorizable types (which have privileged behavior such as math function support and value-preserving conversion rules) and user-defined vectorizable types, which have different rules. The trait-based constraints extend the set of vectorizable types, they do not redefine it.

All existing built-in vectorizable types satisfy the trait-based constraints, so the two categories are consistent. We change only which types are allowed; operator behavior remains element-wise application as currently specified. User-defined vectorizable types work exactly like built-in vectorizable types of the same size, where an operation is available for `vec<T>` if and only if it exists for `T`.

We did notice that it will be necessary to tighten the wording of some operator constraints to explicitly require appropriate return types for user-defined types. This prevents certain classes of errors and performance traps. The constraints distinguish between promotable types (arithmetic types and unscoped enumerations, which may undergo integer promotion) and all other vectorizable types (including `complex`, scoped enumerations, `std::byte`, and user-defined types), which should return the exact type. For example, `uint8_t + uint8_t` produces an `int` due to integer promotion, requiring lenient checking that allows explicit conversion back. In contrast, user-defined vectorizable type operators must return the correct type directly to prevent subtle bugs. This doesn't affect existing built-in vectorizable types, but ensures user-defined vectorizable types behave correctly.

Everything else in the proposal stays the same. All operations and their semantics, performance characteristics, ABI selection, and existing code remain unchanged.

2.4. Scope and Future Directions

2.4.1. In Scope: Element-wise Semantics

This proposal maintains exact semantic parity with existing `simd` operations. All operators require operands of `simd<T>` and return `simd<T>` (or `simd_mask<T>` for comparisons), exactly as `simd<int>` does today. The only change is expanding which types `T` are permitted as elements in a `simd` by adding `std::byte` to the built-in vectorizable types, and introducing trait-based constraints for user-defined vectorizable types.

This design immediately enables important use cases:

- Type-safe dimensional types that maintain scalar semantics
- Enumeration processing
- `std::byte` for binary data processing
- Domain-specific numeric types (saturating, fixed-point, custom precision)
- Future numeric types (bfloat16, float8, and other emerging formats)

Beyond user-defined types, the trait-based approach future-proofs `simd` for numeric type evolution. Compiler builtins or emerging standard types like `std::bfloat16_t` or `std::float8_t`, and vendor-specific formats automatically work without requiring standard amendments. As hardware evolves for machine learning and scientific computing, new numeric types integrate seamlessly into `simd`.

The trait-based gatekeeper change provides substantial value independently, enabling these use cases without requiring the committee to solve significantly harder problems.

2.4.2. Deliberately Out of Scope: Heterogeneous Operations

Heterogeneous type operations, where `simd<A> op simd<B> -> simd<C>`, are deliberately excluded from this proposal. This proposal is purely additive in which types are permitted as elements of `simd` and doesn't change their operator semantics. All `simd` operations remain homogeneous, where `simd<T> op simd<T> -> simd<T>`, exactly as they are for `simd<int>` today.

Heterogeneous operations would require changing `simd` itself and would introduce fundamentally different design problems that do not arise in this proposal:

- **Type-level computation:** The result type of `simd<Length> / simd<Time>` must be computed as `simd<Speed>`. This requires a type algebra that is defined by a units library, not by `simd`. Different units libraries may define different type algebras.
- **Operator template explosion:** Every binary operator would need to be templated over pairs of element types, with constraints determining which combinations are valid. This is a significant specification and implementation burden.
- **ABI reconciliation:** If the input types have different elements then it isn't clear what conversions should take place to resolve them (e.g., what result type to produce from a mixed-type operator). This is a policy decision that belongs to the type algebra (e.g., a units library), not to `simd`. Note that this is consistent with current practice where `simd<int> + simd<double>` is not directly supported either, but can be achieved through conversion of one of its inputs. This proposal maintains that existing behaviour.
- **Ownership of semantics:** The meaning of `Meters / Seconds -> Speed` is defined by a units library such as mp-units ([P3045R4]), not by `simd`. It would be inappropriate for `simd` to encode or assume any particular type algebra. The units library should control these semantics, and may even choose to have a quantity whose type was a `simd`, rather than a `simd` whose element type is a quantity.

NOTE: Conversions between `simd` types (e.g., `simd<Feet>` to `simd<Meters>`) do not constitute heterogeneous operations. A conversion produces a new `simd` of the target type by construction, not a binary operation. Conversions use `static_cast` element-wise, delegating to whatever scalar conversion the type author has defined. This requires no type algebra and no changes to `simd`'s operation semantics. Users can convert first and then operate, achieving heterogeneous workflows through composition of homogeneous operations.

Heterogeneous `simd` operations are a topic that merits its own proposal, ideally authored in collaboration with domain experts in dimensional analysis and units libraries who understand the requirements. Such a proposal would apply to all element types (including arithmetic types such as `simd<int> + simd<double>`), not just user-defined ones.

2.4.3. Forward Compatibility

The current design is fully forward-compatible with future heterogeneous operations. Adding template overloads such as:

```
template<typename T, typename U, typename Abi>
friend basic_vec</* computed result type */, Abi>
operator+(const basic_vec<T, Abi>&, const basic_vec<U, Abi>&);
```

would not conflict with existing homogeneous operators - it would simply add new overloads to the existing set. The trait-based vectorizable definition in this proposal works unchanged with such future extensions.

§ 2.4.4. Future Extension: Heterogeneous Operations

A future proposal could extend `simd` to support heterogeneous operations where operand and result types differ. Such a proposal should be developed in collaboration with the authors of quantities and units libraries (e.g., [P3045R4]) and would need to address type algebra, ABI reconciliation, and the question of which library (`simd` or the units library) owns the semantics of cross-type operations. Similar considerations apply to other standard library types such as `std::chrono::duration` and to linear algebra types, where operations may produce different result types.

§ 3. Motivation

The current restriction to a closed list of vectorizable types prevents several valuable use cases that would naturally benefit from SIMD parallelism, including strong typedefs for physical units, enumerations for state machines and flags, `std::byte` for low-level data processing, and small compound types for structure-of-arrays patterns. This section presents motivating examples.

§ 3.1. Type Safety and Strong Typedefs

Physical units, identifiers, and other domain-specific types are commonly wrapped in strong typedefs to prevent semantic errors:

```
struct Meters { float value; };
struct Seconds { float value; };

// Type safety at scalar level
Meters distance{100.0f};
Seconds time{5.0f};
// Meters m = time; // Error: type mismatch

// Same type safety should extend to parallel code
vec<Meters> distances = {100.0f, 200.0f, 150.0f, 180.0f};
vec<Seconds> times = {5.0f, 10.0f, 7.5f, 9.0f};
// vec<Meters> m = times; // Should also be error
```

Currently users who wish to put these strong types into a `basic_vec` would need to unpack them to `vec<float>`, losing type safety precisely where parallel operations occur. This proposal preserves type safety uniformly.

§ 3.2. Signal and Media Processing Types

Specialized domains use custom numeric types optimized for their workloads:

```
// Fixed-point arithmetic for digital signal processing
struct fixed_point_16s8 {
    std::int16_t data;

    fixed_point_16s8 operator+(fixed_point_16s8 rhs) const {
        return fixed_point_16s8{saturate_add(data, rhs.data)};
    }
    // Other operators...
};

// Should work with vec
vec<fixed_point_16s8> samples = load_audio_samples();
auto processed = apply_filter(samples); // Element-wise fixed-point operations
```

The proposal allows `std::simd` to provide its parallel infrastructure (loads, stores, masking, permutations, reductions) while deferring arithmetic to the user-defined type's operators.

§ 3.3. Enumerations

Enumerations are essentially only restricted integer types with named values. They are widely used for state machines, flags, and encoded data. Vectorizing enumerations enables batch processing of such data.

```
enum class Color : std::uint32_t { Red, Green, Blue, Alpha };

vec<Color> pixel_channels = /* ... */;
auto masked = pixel_channels & Color::Alpha; // Bitwise operations on scoped enums
```

Scoped enums (`enum class`) only allow operations that are valid for the enum itself (typically bitwise operations, comparisons, and conversions), while unscoped enums allow arithmetic operations through implicit conversion to their underlying type. The element-wise application mechanism automatically respects these restrictions without any special handling.

§ 3.4. `std::byte`

`std::byte` is a distinct type representing raw byte data, commonly used in low-level programming. Vectorizing `std::byte` enables efficient byte-level operations such as encryption, checksums, and encoding.

```
vec<std::byte> data = /* load from buffer */;
auto encrypted = data ^ vec<std::byte>{0xFF}; // XOR cipher
```

§ 3.5. Compound Types

Small compound types that fit in 16 bytes can be vectorized as atomic units, enabling structure-of-arrays patterns, or packet processing of multiple values simultaneously.

```
// Coordinate pairs
vec<std::pair<int, int>> coordinates;

// RGBA color pixels
vec<std::array<std::uint8_t, 4>> pixels;
```

§ 4. Understanding Type Constraints

To ensure user-defined types work correctly with `std::simd`, we impose constraints that match hardware capabilities and prevent subtle bugs. In summary the constraints are:

- Trivially copyable
- Size: must be 1, 2, 4, 8, or 16 bytes
- Opt-out mechanism via `disable_vectorization`
- Banned standard library types and categories (pointers, unions, cv-qualified, empty)

We now look in more detail at each of these constraints.

§ 4.1. Trivially Copyable Constraint

We require `std::is_trivially_copyable_v<T>`. Many `std::simd` operations move elements bitwise (permutations, broadcasts, gathers, scatters). For these to work correctly, an element's value must be preserved when its bit pattern is copied. Trivially copyable types have no special copy, move, or destroy logic, so bitwise copying always produces correct results.

4.2. Size Constraint

We require `sizeof(T)` to be exactly 1, 2, 4, 8, or 16 bytes. All known hardware vector instruction sets support only power-of-2 element sizes. No shipping or announced vector ISA supports non-power-of-2 element widths. The largest current vectorizable type is `std::complex<double>` at 16 bytes.

An alternative design considered was to define the valid sizes as implementation-defined or derived from the sizes of existing vectorizable types. However, explicitly listing the sizes is simpler, and directly reflects the reality of all current hardware. If future hardware or standard types introduce new element widths, the list can be extended by a future standard revision — a straightforward, non-breaking change.

4.3. Padding and Bit Representation

SIMD operations treat element types as uninterpreted bit patterns of the specified size. If a user-defined type contains padding bytes (e.g., `struct ThreeChars { char a, b, c; }` typically has `sizeof=4` with one padding byte), `simd` is agnostic to which bits represent data versus padding. All bits are preserved through operations, with semantics determined solely by the element type's operators. This is consistent with trivially copyable semantics.

4.4. Opt-Out Mechanism

The standard library uses a common pattern for selectively disabling features where a variable template can be specialized. For `std::simd`, this proposal adds `std::disable_vectorization<T>`, which will default to `false` but can be specialized to `true` for types that should not be vectorizable. This mechanism will allow the implementation to opt out of allowing vectorization for semantically inappropriate types which otherwise appear to permit vectorization.

Users may specialize `disable_vectorization` for their own types, such as:

```
namespace my_lib {
    struct InternalType { std::uint64_t data; };
}

template<>
inline constexpr bool std::disable_vectorization<my_lib::InternalType> = true;
```

Specializations for cv-qualified or reference types are ill-formed.

4.5. Banned Standard Library Types

In addition to allowing the user to opt out of some types, the mechanism can also be used by the implementation to ban specific standard types and categories which have no meaningful vectorization semantics.

Type categories automatically banned:

- **Pointer types** (`is_pointer_v` or `is_member_pointer_v`): The direction of C++ is toward stronger type safety and away from raw pointer manipulation, as reflected in the adoption of `std::span` ([P0122R7]) and ongoing memory safety work ([P3081R0]). A `simd` of pointers would hold semantically unrelated addresses where element-wise arithmetic has no coherent meaning. The common use case of non-contiguous memory access is already well-served by storing integer indices in a `simd` and using gather/scatter operations against a base `std::span`, which is both safer and maps directly to hardware instructions.
- **Union types** (`is_union_v<T>`): Ambiguity about which member is active.
- **CV-qualified types** (`is_const_v<T>` or `is_volatile_v<T>`): Breaks assignment operators. (Note: cv-qualified vector objects like `const vec<int>` are permitted; the ban applies only to cv-qualified element types.)
- **bool**: Overlaps semantically with `basic_mask`, which is the intended mechanism for parallel boolean values.
- **Empty types** (`is_empty_v<T>`): Carry no data.

Standard library types:

```
// Types not caught by category rules
template<> inline constexpr bool disable_vectorization<std::nullptr_t> = true;
template<> inline constexpr bool disable_vectorization<std::source_location> = true;
template<class T, class Abi>
inline constexpr bool disable_vectorization<std::basic_vec<T, Abi>> = true;
template<class T, class Abi>
inline constexpr bool disable_vectorization<std::basic_mask<T, Abi>> = true;
```

Note that under these constraints, arrays (`int[4]`), `std::pair`, and `std::tuple` are not banned, provided they satisfy the constraints. They can all be useful in their own way, such as representing vector-processing of packet processing patterns, structured data, and structure-of-array layouts. Even if these types do not provide arithmetic or mathematical operations, it is still useful to be able to use them for parallel load/store, masking, permutation and bit-level operations. Note also that the category rules catch many of the standard types that are disallowed (e.g., `std::monostate` is excluded by being an empty type).

This list is not exhaustive; implementations may provide additional specializations for other types where vectorization is semantically inappropriate.

§ 4.6. `std::byte` as a built-in vectorizable type

Although `std::byte` is technically a scoped enumeration (`enum class byte : unsigned char {}`), this proposal adds it to the built-in vectorizable type list rather than treating it as a user-defined vectorizable type. This is justified because:

- It is defined by the standard library, not by users
- It has dedicated language support, including `to_integer<>()` in `[cstddef.syn]`
- Its operators (`<<`, `>>`, `|`, `&`, `^`, `~`) are explicitly specified by the standard library
- The implementation knows its exact semantics. There is nothing to infer through element-wise application

In these respects `std::byte` is analogous to `std::complex`: a standard library type with fixed, known semantics that the implementation can handle directly. Treating it as built-in ensures that implementations use optimized code paths for `std::byte` operations and never consult ADL customization points, consistent with how other built-in vectorizable types are handled.

§ 4.7. Summary of Constraints

The constraints work together to ensure types are safe and efficient for vectorization:

- **Trivially copyable** enables bitwise element manipulation
- **Power-of-2 size** matches hardware vector capabilities
- **Opt-out mechanism** allows excluding inappropriate types

These enable user-defined types like `vec<Meters>`, `vec<Color>`, and `vec<std::array<uint8_t, 4>>`, while excluding pointers, unions, cv-qualified types, empty types, and opted-out types.

§ 5. Operations on User-Defined Types

This section describes how `std::simd` operations work with user-defined element types. The key principle is element-wise application: operations on `vec<T>` apply the corresponding operation on `T` to each element independently.

User-defined types are treated as atomic blocks of bits whose internal structure is not modified by `simd` operations. This proposal does not include struct-of-arrays conversions or layout transformations for user-defined types.

5.1. Operator Constraints

The `std::simd` specification provides operators conditionally using `requires` clauses. The working draft currently checks only that element-wise operations are valid expressions, without constraining return types. This proposal tightens these constraints to require appropriate return types, with different rules for promotable types versus all other vectorizable types.

For **promotable vectorizable types** (arithmetic types excluding `bool`, and unscoped enumerations) operators may return a promoted type (e.g., `uint8_t + uint8_t → int`), which is then explicitly converted back to the element type. This preserves existing behavior for built-in types.

For **all other vectorizable types** (scoped enumerations, `std::byte`, `std::complex`, and user-defined types), operators must return exactly `value_type`. This prevents subtle bugs where user-defined operators return incorrect types.

The constraints use exposition-only concepts that capture this two-tier checking:

```
template<typename T, typename BinaryOp>
concept supported-binary-op = /* exposition only */
    (is_arithmetic_v<T> || (is_enum_v<T> && !is_scoped_enum_v<T>)) ? // Is it promotable?
    requires(T a, T b) { BinaryOp{}(a, b); } :
    requires(T a, T b) { { BinaryOp{}(a, b) } -> same_as<T>; };
```

Return type requirements:

Arithmetic operators (`+`, `-`, `*`, `/`, `%`, `&`, `|`, `^`, `<<`, `>>`, unary `-`, `~`):

- For promotable types (arithmetic types excluding `bool`, and unscoped enums): Allow promotion with explicit conversion back
- For all other vectorizable types: Must return exactly `value_type`

Comparison operators (`==`, `!=`, `<`, `<=`, `>`, `>=`):

- Must return `bool` (no promotion of result type)

These requirements prevent size mismatches, avoid conversions that change semantics, and prevent performance traps from proxy types, while maintaining backward compatibility for arithmetic types.

NOTE: Comparison operators are not synthesized from each other, maintaining parity with existing `simd` behavior for built-in vectorizable types. For example, `operator!=` is not synthesized from `operator==`. This avoids introducing inconsistency with current `simd` semantics. Synthesis of comparison operators could be proposed separately as an enhancement to all `simd` types, not just user-defined ones.

Examples:

```
struct Meters {
    float value;
    Meters operator+(Meters rhs) const { return Meters{value + rhs.value}; }
    bool operator<(Meters rhs) const { return value < rhs.value; }
};

vec<Meters> a, b;
auto sum = a + b;           // ✓ OK: operator+ returns Meters
auto mask = a < b;          // ✓ OK: operator< returns bool

struct NoAdd { float value; };
vec<NoAdd> x, y;
auto result = x + y;        // ✗ Error: operator+ not defined

struct DifferentReturn {
    int16_t value;
    int32_t operator+(DifferentReturn) const; // Change return type
```

```
};
vec<DifferentReturn> v, w;
auto bad = v + w; // ❌ Error: int32_t is not DifferentReturn
```

Compound assignments use the same constraints as their corresponding binary operators:

```
friend constexpr basic_vec& operator+=(basic_vec& lhs, const basic_vec& rhs)
    requires supported_binary_op<value_type, plus<>>; // Same as operator+
```

All six comparison operators continue to be independently specified.

The mask type `basic_mask<value_type, Abi>` is determined by the element type's size, not its contents. Masks indicate active/inactive lanes for a group of bits of size `sizeof(value_type)`. For any user-defined type, the mask semantics are identical to those of built-in vectorizable types of the same size - one mask bit per element, regardless of what data the element contains.

5.2. Conversions and Casts

Converting constructors use `static_cast` for element conversion:

```
//Element `i` is initialized with `static_cast<T>(v[i])`.
template<typename U>
explicit constexpr basic_vec(const basic_vec<U, Abi>& v)
    requires /* appropriate constraints */;
```

This naturally supports user-defined conversions:

```
struct Meters { float value; };
struct Feet {
    float value;
    operator Meters() const { return Meters{value * 0.3048}; }
};

vec<Feet> feet = {3.0f, 6.0f, 9.0f, 12.0f};
vec<Meters> meters{feet}; // ✅ Works via conversion operator
```

The existing `static_cast` semantics handle all conversion scenarios without additional specification.

5.2.1. Value-Preserving Conversions

The working draft defines "value-preserving" only for conversions from arithmetic types: *"The conversion from an arithmetic type U to a vectorizable type T is value-preserving if all possible values of U can be represented with type T "* ([\[simd.general\]](#)). This definition is precise for arithmetic types but does not extend to user-defined types.

For conversions involving user-defined types, this proposal defers to the type author's judgment as expressed through implicit versus explicit conversions:

For conversions between built-in vectorizable types: Use the existing value-preserving definition (e.g., `int` to `long` is value-preserving, but `double` to `float` is not).

For conversions involving at least one user-defined type: Use `std::is_convertible_v<From, To>` to determine if the conversion may be implicit:

- If `is_convertible_v<From, To>` is `true`, the type author has declared the conversion safe via an implicit constructor, so `simd` allows it implicitly
- If `is_convertible_v<From, To>` is `false` but `is_constructible_v<To, From>` is `true`, the type author requires `explicit`, so `simd` also requires explicit conversion

Examples:

```
struct Meters {
    float value;
    Meters(float f) : value(f) {} // Implicit - author says it's safe
};

struct Feet {
    float value;
    explicit Feet(float f) : value(f) {} // Explicit - author says be careful
};

vec<float> vf = {...};

vec<Meters> v0 = vf;           // OK - Meters(float) is implicit
vec<Feet> v1 = vf;            // Error - Feet(float) is explicit
vec<Feet> v2 = vec<Feet>(vf); // OK - explicit construction

std::span<float, 1024> sf;

// OK - implicit conversion from float to Meters
auto m_vec = unchecked_load<vec<Meters, 8>>(sf);

// Error - implicit conversion from float to Feet not allowed
auto f_vec = unchecked_load<vec<Feet, 8>>(sf);

// OK - conversion from float allowed with flag_convert tag
auto f_vec = unchecked_load<vec<Feet, 8>>(sf, flag_convert);
```

This approach:

- Respects the type author's design decisions about safety
- Maintains consistency with scalar usage patterns
- Avoids second-guessing the type author's judgment
- Does not require `simd` to define value-preservation semantics for user-defined types

Same-type operations are unaffected - `simd<Meters>(Meters{3.14f})` broadcasts by copying, not converting, so these rules don't apply.

Note that a type author could declare an implicit conversion that loses information (e.g., `Meters(double)` with `float` storage). However, this is the type author's choice at the scalar level, and `simd` should not override that judgment. If the scalar user-defined type allows implicit lossy conversion, `simd` does too.

5.3. Maths Functions

The working draft provides mathematical functions such as `sin`, `cos`, `sqrt`, and `abs` for `basic_vec`, but constrains them to built-in floating-point and integer types. This proposal does not extend these functions to user-defined types. Unlike operators, which map directly to simple scalar operations that compilers can reliably auto-vectorize, maths functions typically involve internal loops, conditionals, and table lookups that would prevent the compiler from producing efficient vectorized code through element-wise inference. Rather than promising best-effort inference that risks a performance cliff, we disallow these functions for user-defined types entirely. Users who need them should provide their own implementations in their type's namespace, where they will be found via ADL at unqualified call sites, following the established pattern of `std::swap`.

`min` and `max` are the exception. They are trivial (typically just a compare and a select) and widely used, making them good candidates for element-wise inference. They are the only mathematical functions for which we provide support for user-defined types. For built-in types, implementations use dedicated hardware instructions (e.g., `vpminsw`, `vpmasw` on x86). For

user-defined types, the implementation applies scalar `min/max` element-wise. If a user needs to provide an optimized vector-level implementation, the same ADL mechanism applies:

```
namespace my_lib {
    struct MyType { int16_t data; /* ... */ };

    // Found via ADL when called as unqualified min(a, b)
    template<typename Abi>
    basic_vec<MyType, Abi>
    min(const basic_vec<MyType, Abi>& a,
        const basic_vec<MyType, Abi>& b) {
        return /* optimized implementation */;
    }
}
```

As with `swap`, users should use unqualified calls to enable ADL. A qualified call to `std::min` will always use the standard element-wise implementation.

5.4. Reductions

Reduction operations (e.g., `reduce`, `reduce_min`, `reduce_max`) apply an operation pairwise across elements:

```
// Applies `binary_op` pairwise to elements in unspecified order.
template<typename T, typename Abi, typename BinaryOp = std::plus<>>
constexpr T reduce(const basic_vec<T, Abi>& v, BinaryOp binary_op = {});
```

For `reduce` with a binary operation such as `std::plus<>`, the operation goes through the standard three-tier dispatch: built-in optimized path, `simd_operator` customization, or element-wise inference. This means user-defined types benefit from customization automatically — if a user provides `simd_operator` for `std::plus<>`, `reduce(v, std::plus<>())` will use it.

NOTE: Reductions assume associativity. For types with non-associative operations, results may differ from sequential left-to-right reduction. This is consistent with floating-point behavior, where `reduce(v, std::plus<>())` may produce different results than sequential summation due to intermediate rounding. The working draft already specifies this behavior via preconditions on the binary operation.

```
struct ModularInt {
    int value;
    ModularInt operator+(ModularInt rhs) const {
        return ModularInt{(value + rhs.value) % 100};
    }
};

vec<ModularInt> v = {50, 30, 40, 20};
auto sum = reduce(v, std::plus<>());
// Result: ModularInt{40}
// Could evaluate as
// ((50+30)+40)+20 = (80+40)+20 = 20+20 = 40
// (50+30)+(40+20) = 80+60 = 40
```

5.4.1. `reduce_min` and `reduce_max`

`reduce_min` and `reduce_max` differ from other reductions. The working draft specifies them in terms of `operator<` ("the value of an element `x[i]` for which `x[i] < x[j]` is false for all `i`"), not in terms of `min/max`. For user-defined types, `operator<` can be customized via `simd_operator` with `std::less<>`, providing a path to optimized reductions.

However, this path is less direct than for other reductions. With `reduce(v, std::plus<>())`, the user customizes `operator+` and the reduction benefits immediately. With `reduce_min`, the user might expect `reduce_min` to use their

custom `min` internally, but the specification doesn't require this connection.

We note that this specification choice conveniently accommodates the masked variants of `reduce_min/reduce_max`, which operate only on active lanes, without requiring masked `min/max` overloads on `basic_vec`.

We considered whether explicit customization points for min/max reductions are needed. The possible approaches are:

- Respecify min/max in terms of compare-and-select, routing through the existing `simd_operator` customization for comparisons. However, this would be a retrograde step for built-in types where dedicated hardware min/max instructions outperform a compare-and-blend sequence.
- Introduce tag-based customization for min/max, analogous to how `simd_operator` uses `std::plus<>`, `std::minus<>`, etc. to identify operations. This would require new transparent function object types such as `std::simd::min_op` and `std::simd::max_op`, and the `simd_operator` dispatch would need to be extended to recognise them. The reductions would then also need to be respecified in terms of a masked form of that customized min/max, adding considerable specification complexity.
- Respecify the reductions in terms of an unqualified call to min/max, which would naturally find user-provided overloads via ADL. This is the simplest approach, but would change how existing reductions on built-in types are specified, which is beyond the intended scope of this proposal.

Given that element-wise inference and ADL-based overloading provide functional (if imperfect) paths today, we defer this question until practical experience demonstrates a need.

5.5. Load and Store Operations

Load operations already specify element conversion via `static_cast`:

```
// Element `i` is initialized with `static_cast<T>(*std::next(first, i))`.
template<typename It>
constexpr basic_vec(It first, It last);
```

This naturally handles both same-type loads and converting loads via the `static_cast` mechanism (see §5.2 [Conversions and Casts](#) for examples). Implementations may optimize by using vector loads followed by vector conversions rather than converting each element individually.

Store operations work similarly. No specification changes are needed.

5.6. Copy Operations

Operations that move elements without interpreting values work on any trivially copyable type:

- `permute`, `broadcast` - rearrange elements
- `compress`, `expand` - conditional packing/unpacking
- `select` - conditional element selection
- `chunk`, `cat` - size/shape changes

These operate at the bit level and require no knowledge of element semantics. The trivially copyable constraint ensures they already work correctly for user-defined types.

5.7. Implementation Considerations

In this section we shall briefly examine two important implementation considerations when supporting user-defined types in `std::simd`: exception safety and ABI selection.

All `basic_vec` operations are declared `noexcept` in the working draft. This has important implications for user-defined types: if an element-wise operation throws an exception during a `simd` operation, `std::terminate` will be called.

This behavior is appropriate for SIMD code. Detecting and propagating exceptions on individual elements would require serializing the operation, checking each element's result, and managing partial completion state. This fundamentally contradicts SIMD's purpose of parallel execution. User-defined types intended for use in `simd` should have non-throwing operations, or accept that exceptions will terminate the program.

The `noexcept` specification means:

- Element-wise operations are not required to be `noexcept` themselves
- If they do throw during `simd` operations, `std::terminate` is called
- Users must ensure their types' operations don't throw in practice
- This is consistent with SIMD being performance-critical code where exceptions are inappropriate

§ 5.8. ABI Selection for User-Defined Types

ABI selection determines the vector width (number of elements) for a `simd` object. For user-defined types, ABI selection is based solely on `sizeof(T)`. A UDT of size N bytes is treated identically to built-in vectorizable types of size N for ABI purposes. This means:

```
struct A { int32_t x; }; // sizeof=4 → treated like int32_t for ABI
struct B { float f; }; // sizeof=4 → treated like float for ABI
struct C { uint8_t data[4] }; // sizeof=4 → treated like int32_t for ABI
```

Any two types with the same size will receive the same ABI and therefore the same number of elements:

```
struct MyInt32 { std::int32_t value; };

vec<int> v1; // Suppose this gets 512-bit vectors = 16 elements
vec<float> v2; // Also 512-bit vectors = 16 elements (both 4 bytes)
vec<MyInt32> v3; // Also 512-bit vectors = 16 elements (also 4 bytes)
```

Implementations select vector width based on element size to match hardware capabilities. This ensures consistent behavior and predictable performance characteristics across types of the same size.

§ 6. Customization Points

In early revisions of this paper, we considered a design where all operations on user-defined types were implemented as customization points discovered via ADL. While allowing very precise user control, it also meant the user always had to provide implementations for every operation, even if the compiler could infer the same operation for itself. In later revisions we switched to a hybrid approach where element-wise inference is the default, and customization points are only consulted when users want to provide optimized implementations for specific operations.

There are two categories of customization: operations (e.g., addition, multiplication) and conversions (e.g., `vec<Feet>` to `vec<Meters>`). Both are discovered via argument-dependent lookup in the namespace of the element type. Users do not inject declarations into namespace `std`. Because `basic_vec<T, Abi>` carries `T`'s associated namespaces, ADL naturally finds overloads declared alongside the element type definition.

§ 6.1. Operation Customization

A single overloaded function name `simd_operator` handles both unary and binary operations, distinguished by arity:

```
// In user's namespace, discovered via ADL:
auto simd_operator(vec<T> v, Op op) -> vec<T>; // Unary
auto simd_operator(vec<T> v1, vec<T> v2, Op op) -> vec<T>; // Binary
```

The `Op` parameter is one of the standard transparent function objects (`std::plus<>`, `std::minus<>`, `std::multiplies<>`, `std::negate<>`, etc.) identifying which operation is being customized ([P4006] proposes adding

`bit_lshift<>` and `bit_rshift<>` to help here). This design allows users to customize individual operations by providing overloads for specific function objects while relying on element-wise inference for everything else.

For arithmetic and bitwise operations, the return type of `simd_operator` must be exactly `basic_vec<value_type, Abi>`. For comparison operations, the return type must be exactly `basic_mask<value_type, Abi>`. If ADL finds a `simd_operator` that returns a different type, it is not considered and element-wise inference is used instead.

When a `simd` operation is performed, the implementation uses a three-tier dispatch:

1. **Built-in vectorizable types:** The implementation uses its own optimized code path. The `simd_operator` customization point is never checked. This ensures that built-in types always use the most efficient implementation and prevents users from accidentally overriding well-optimized library code.
2. **ADL `simd_operator`:** For user-defined vectorizable types, if a valid `simd_operator` overload is found via ADL, it is used.
3. **Element-wise fallback:** If no `simd_operator` is found, the implementation applies the scalar operator to each element independently, relying on compiler auto-vectorization.

In conceptual terms, the dispatch looks like this:

```
template<typename T> // Built-in vectorizable types
requires /*built-in-vectorizable*/ && supported_binary_op<T, std::plus<>>
friend basic_vec operator+(const basic_vec& lhs, const basic_vec& rhs) {
    return /* implementation-defined optimized implementation */;
}

template<typename T> // User-defined vectorizable types
requires /*user-defined-vectorizable*/ && supported_binary_op<T, std::plus<>>
friend basic_vec operator+(const basic_vec& lhs, const basic_vec& rhs) {
    if constexpr (requires { simd_operator(lhs, rhs, std::plus<>{}); }) {
        return simd_operator(lhs, rhs, std::plus<>{}); // Custom via ADL
    } else {
        return /* element-wise application */; // Default
    }
}
```

For enumerations and user-defined types without customization, the `simd_operator` check fails at compile time and element-wise inference is used. Since enumerations without custom operators compile to simple integer arithmetic, element-wise inference produces optimal code.

Example: Providing a custom saturating add that maps directly to hardware instructions:

```
namespace my_lib {
    struct saturating_int16 {
        std::int16_t data;

        friend saturating_int16 operator+(saturating_int16 lhs, saturating_int16 rhs) {
            auto r = std::int32_t(lhs.data) + std::int32_t(rhs.data);
            return saturating_int16{std::clamp<int32_t>(r, -32768, 32767)};
        }
    };

    // Custom SIMD addition using native saturating instructions
    template<typename Abi>
    basic_vec<saturating_int16, Abi>
    simd_operator(const basic_vec<saturating_int16, Abi>& lhs,
                  const basic_vec<saturating_int16, Abi>& rhs,
                  std::plus<>) {
        // Implementation can use platform-specific intrinsics
        // e.g., _mm256_adds_epi16 on x86
        return /* optimized saturating add */;
    }
}
```

Without the customization, element-wise inference would apply the scalar `operator+` to each element. As shown in the implementation experience section (§ 7 Implementation Experience), leading compilers can often auto-vectorize such operations into the same hardware instructions. The customization point provides a guarantee of optimal code generation regardless of compiler sophistication.

6.2. Conversion Customization

Conversions between `simd` types use a separate customization point, `simd_convert`, with a tag-based dispatch pattern. The tag type `convert_to_t<T>` and its associated variable template are provided as part of the public API:

```
// Provided by the simd library:
template<typename T>
struct convert_to_t {
    using type = T;
    constexpr explicit convert_to_t() noexcept = default;
};

template<class T> inline constexpr convert_to_t<T> convert_to{};
```

Users provide overloads of `simd_convert` in the namespace of their element type:

```
// User customization point signature:
template<typename Abi>
basic_vec<To, Abi> simd_convert(const basic_vec<From, Abi>& source, convert_to_t<To>);
```

The `convert_to_t<To>` tag argument serves two purposes: it enables ADL discovery, and it allows users to write customization points for specific conversion directions.

As with operations, conversion dispatch uses a three-tier strategy:

1. **Both built-in vectorizable types:** The implementation uses its own optimized conversion. The `simd_convert` customization point is never checked.
2. **ADL `simd_convert`:** If at least one type is not a built-in vectorizable type and a valid `simd_convert` overload is found via ADL that returns exactly `basic_vec<To, Abi>`, it is used.
3. **Element-wise fallback:** If no `simd_convert` is found, the implementation falls back to element-wise `static_cast`, which invokes the scalar conversion operators or constructors on each element.

Example: Optimizing BFloat16 to float conversion using hardware instructions:

```
namespace my_lib {
    struct BFloat16 { uint16_t bits; /* ... */ };

    template<typename Abi>
    basic_vec<float, Abi>
    simd_convert(const basic_vec<BFloat16, Abi>& source, convert_to_t<float>) {
        #ifdef __AVX512BF16__
            return /* use native bfloat16 conversion instructions */;
        #else
            return /* software shift-based implementation */;
        #endif
    }
}
```

§ 7. Implementation Experience

We implemented this approach in Intel's `std::simd` implementation and tested across multiple Intel architectures. This section presents the technical details: code generation results, assembly analysis, and identified limitations.

§ 7.1. Test Implementation

We experimented with a number of different test types, including an enumeration, a strong type, and a saturating integer type to evaluate code generation quality:

```
enum Color {Red, Green, Blue};

struct Meters {
    float value;

    Meters operator+(Meters rhs) const { return Meters{value + rhs.value}; }
    bool operator<(Meters rhs) const { return value < rhs.value; }
};

struct saturating_int16 {
    saturating_int16(int v) : data(v) {}
    std::int16_t data;

    // Saturating addition
    friend saturating_int16 operator+(saturating_int16 lhs, saturating_int16 rhs) {
        auto r = std::int32_t(lhs.data) + std::int32_t(rhs.data);
        return saturating_int16(std::clamp<int32_t>(r, -32768, 32767));
    }

    friend bool operator>(saturating_int16 lhs, saturating_int16 rhs) {
        return lhs.data > rhs.data;
    }

    // Other operators defined similarly...
};
```

§ 7.2. Successful Inference Cases

Testing was performed with Clang 20 and Intel oneAPI 2025.0 targeting Intel Sapphire Rapids. For most operations, these compilers generated excellent code from element-wise operator application. The generated assembly uses native vector instructions throughout, with no scalar fallback or element-by-element processing. The instruction selection matches what hand-written intrinsics would produce, demonstrating that element-wise inference can generate performance-competitive code for common operations.

Important note on compiler variance: Optimization quality for user-defined types varies significantly between compiler vendors and versions. The results presented here reflect what's possible with current leading implementations - other compilers may produce substantially less optimal code, particularly for complex operations like reductions. This variance is a quality-of-implementation issue, not a fundamental limitation of the design. Clang and oneAPI demonstrate the approach works. Compilers that currently struggle will improve over time as their optimization passes mature. Users should verify code quality with their specific toolchains and consider using the customization mechanisms ([§6 Customization Points](#)) if their compiler doesn't yet optimize well.

See [§13 Appendix: Assembly Code Examples](#) for detailed assembly listings showing the code generated for a variety of common patterns.

§ 7.3. Identified Limitation

We did identify one case where element-wise inference produced suboptimal code:

C++ Code	Generated Assembly (Suboptimal)
<pre> auto reduce_add(vec<saturating_int16> v) { return reduce(v, std::plus<{}>); } </pre>	<pre> reduce_add(...): vextracti128 xmm1, ymm0, 1 vpaddsw xmm0, xmm0, xmm1 vpextrq rdx, xmm0, 1 vmovq rax, xmm0 mov rsi, rax shr rsi, 48 mov rcx, rdx shr rcx, 48 lea edi, [rsi + rcx] movsx edi, di sar edi, 15 xor edi, -32768 add si, cx cmovo esi, edi // ... continues with scalar operations </pre>

For this reduction, the compiler started with vector operations but then switched to element-by-element scalar execution. The first two instructions are correct (extract and vector add), but subsequent operations process elements individually rather than maintaining vectorization throughout.

7.4. Implications for Customization

This experience demonstrates that:

1. Element-wise inference succeeds for most operations with leading compilers: Permutations, broadcasts, and direct operators generate optimal code with current Clang and Intel oneAPI implementations.
2. Compiler maturity varies significantly: Optimization quality for user-defined types shows substantial differences between compiler vendors and versions. While Clang and oneAPI generate excellent code, other compilers may produce significantly less optimal results - sometimes falling back to scalar operations where vectorization should succeed. This reflects differences in compiler optimization sophistication, not limitations of the design itself.
3. Specific limitations exist: Even with mature compilers, complex algorithms like reductions may not auto-vectorize perfectly from scalar operator definitions.
4. Customization provides value: For cases where compilers struggle, the ADL-based customization mechanism (`simd_operator` and `simd_convert`) enables users to provide optimized implementations, ensuring good performance regardless of compiler optimization quality.

The identified limitations motivated the customization design presented in [§ 6 Customization Points](#). However, these limitations do not diminish the value of the core proposal's element-wise inference and the customization mechanism also serves as both a performance optimization for complex cases and a portability tool for users working with compilers that haven't yet achieved sophisticated UDT vectorization.

7.5. Implementation Impact

Implementations already handle element types generically for many operations (permutations, broadcasts, masking). The trait-based definition formalizes this practice and extends it uniformly.

The following changes are needed:

- Modify type trait checking for the `vectorizable` concept/trait
- Add `disable_vectorization` variable template with standard library specializations
- Update operator constraints to check return types (constraints already present, only need tightening)
- Add `convert_to_t` and `convert_to` tag types
- Add customization point dispatch logic for user-defined vectorizable types

The effort to customize the implementation is minimal. The core machinery already exists and only the gate-keeping logic and dispatch logic changes. The implementation experience demonstrates the approach described in this proposal is viable.

Technical details and example implementation are provided in [§ 12 Appendix: Customization Point Technical Details](#).

§ 8. Extended Enum and Byte Support

With our proposal, enumerations and `std::byte` now become vectorizable. Consequently, related utility functions should be extended to work with `simd`:

```
// Element-wise to_underlying for enumerations
template<class Enum, class Abi>
constexpr rebind_t<underlying_type_t<Enum>, basic_vec<Enum, Abi>>
    to_underlying(const basic_vec<Enum, Abi>& v) noexcept;

// Element-wise to_integer for std::byte
template<class IntegerType, class Abi>
constexpr rebind_t<IntegerType, basic_vec<byte, Abi>>
    to_integer(const basic_vec<byte, Abi>& v) noexcept;
```

These provide consistency with their scalar counterparts and convenience for common conversions.

§ 9. Proposed Wording

The wording in this section is relative to the working draft at <https://eel.is/c++draft/simd>.

§ 9.1. Modify [simd.syn]

Add the following declarations to [simd.syn]:

```
// [simd.convert.tag], customization point conversion tag types
template<typename T>
struct convert_to_t;

template<class T>
inline constexpr convert_to_t<T> convert_to{};

// [simd.disable], disabling customization point vectorization
template<class T>
inline constexpr bool disable_vectorization = see below;
```

§ 9.2. Modify [simd.general]

Modify [simd.general] as follows:

The set of *vectorizable types* comprises

- all standard integer types, character types, and the types `float` and `double`;
- `std::float16_t`, `std::float32_t`, and `std::float64_t` if defined;
- `complex<T>` where `T` is a vectorizable floating-point type `T`;
- `std::byte`; and
- any type `T` for which:
 - `is_trivially_copyable_v<T>` is true,
 - `sizeof(T)` is 1, 2, 4, 8, or 16, and
 - `disable_vectorization<T> ([simd.disable])` is false.

The types in the first four bullets are the *built-in vectorizable types*. Types that are vectorizable only by virtue of the fifth bullet are *user-defined vectorizable types*. [Note: All built-in vectorizable types satisfy the trait-based constraints of the fifth bullet. A type that appears in the first four bullets is a built-in vectorizable type regardless of any specialization of `disable_vectorization`. —end note]

9.3. Add [simd.convert.tag]

Insert a new subclause [simd.convert.tag]:

9.3.1. Conversion tag types [simd.convert.tag]

```
template<typename T>
struct convert_to_t {
    using type = T;
    constexpr explicit convert_to_t() noexcept = default;
};

template<class T>
inline constexpr convert_to_t<T> convert_to{};
```

The class template `convert_to_t` and the variable template `convert_to` serve as tag types for the `simd_convert` customization point ([simd.cust.convert]).

9.4. Add [simd.disable] after [simd.general]

Insert a new subclause [simd.disable] after [simd.general]:

9.4.1. Disabling vectorization [simd.disable]

```
template<class T>
inline constexpr bool disable_vectorization = see below;
```

The variable template `disable_vectorization<T>` evaluates to `true` if any of the following conditions hold:

- `is_pointer_v<T>` is true, or
- `is_member_pointer_v<T>` is true, or
- `is_union_v<T>` is true, or
- `is_same_v<remove_cvref_t<T>, T>` is false, or
- `is_empty_v<T>` is true, or
- `is_same_v<T, bool>` is true, or
- A program-defined or implementation-provided specialization of `disable_vectorization<T>` explicitly sets it to true.

Otherwise, `disable_vectorization<T>` evaluates to `false`.

A program may provide explicit specializations of `disable_vectorization` for program-defined types. Such specializations shall be usable in constant expressions and have type `constexpr bool`.

Specializations of `disable_vectorization` for cv-qualified types or reference types are ill-formed.

The implementation provides explicit specializations that set `disable_vectorization` to `true` for the following standard library types: `nullptr_t`, `source_location`, `basic_vec<T, Abi>`, and `basic_mask<T, Abi>`.

Implementations may provide additional specializations for other types where vectorization is semantically inappropriate.

9.5. Add exposition-only concepts to [simd.expos]

Add the following to [simd.expos], after the existing exposition-only definitions:

```
template<typename T>
concept promotable-type = // exposition only
    (is_arithmetic_v<T> && !is_same_v<T, bool>) ||
    (is_enum_v<T> && !is_scoped_enum_v<T>);

template<typename T, typename UnaryOp>
concept supported-unary-op = // exposition only
    promotable-type<T> ?
        requires(T a) { UnaryOp{}(a); } :
        requires(T a) { { UnaryOp{}(a) } -> same_as<T>; };

template<class T, class BinaryOp>
concept supported-binary-op = // exposition only
    ( promotable-type<T> && requires(T a, T b) { BinaryOp{}(a, b); } ) ||
    (!promotable-type<T> && requires(T a, T b) { { BinaryOp{}(a, b) } -> same_as<T>; });
```

[Note: The *promotable-type* concept identifies types that participate in C++'s standard implicit conversion and integer promotion rules (arithmetic types excluding `bool`, and unscoped enumerations). For these types, binary operations may return a promoted type that requires explicit conversion back to `value_type` (e.g., `uint8_t + uint8_t` returns `int`). For all other vectorizable types (scoped enumerations, `byte`, `complex`, and user-defined types), operations must return exactly `value_type`. `bool` is excluded from *promotable-type* because it is banned as an element type via `disable_vectorization` ([simd.disable]). —end note]

§ 9.6. Add [simd.cust] — Customization points

Insert a new subclause **[simd.cust]**:

§ 9.6.1. Customization points [simd.cust]

Customization points allow users to provide optimized implementations of operations and conversions for user-defined vectorizable types. Customization points are discovered via argument-dependent lookup ([basic.lookup.argdep]) in the namespaces associated with the element type.

[*Note:* Because `basic_vec<T, Abi>` carries the associated namespaces of `T`, ADL naturally finds overloads declared alongside the element type definition. Users do not inject declarations into namespace `std`. — *end note*]

Customization points are never consulted for built-in vectorizable types. For built-in vectorizable types, the implementation always uses its own optimized code paths.

[*Note:* Implementations are encouraged to emit a diagnostic if a `simd_operator` or `simd_convert` overload is declared that would never be consulted because it targets a built-in vectorizable type. — *end note*]

§ 9.6.1.1. Operation customization [simd.cust.op]

For user-defined vectorizable types, when a unary or binary arithmetic or bitwise operation is performed on a `basic_vec`, the implementation determines the result as follows:

1. If the expression `simd_operator(v, op)` (for unary) or `simd_operator(v1, v2, op)` (for binary) is well-formed via ADL, where `op` is an object of the corresponding standard transparent function object type, and the return type is exactly `basic_vec<value_type, Abi>`, then the result is that expression.
2. Otherwise, the result is the element-wise application of the scalar operator.

For user-defined vectorizable types, when a comparison operation is performed on a `basic_vec`, the implementation determines the result as follows:

1. If the expression `simd_operator(v1, v2, op)` is well-formed via ADL, where `op` is an object of the corresponding standard transparent function object type (`equal_to<>`, `not_equal_to<>`, `less<>`, `less_equal<>`, `greater<>`, `greater_equal<>`), and the return type is exactly `basic_mask<value_type, Abi>`, then the result is that expression.
2. Otherwise, the result is the element-wise application of the scalar comparison operator.

[*Note:* The well-formedness checks above are SFINAE-friendly. If ADL finds a `simd_operator` overload with an incorrect return type, it is not considered and the implementation falls through to element-wise application. — *end note*]

The `Op` parameter is one of the following standard transparent function objects:

- Unary: `negate<>`, `bit_not<>`
- Binary arithmetic: `plus<>`, `minus<>`, `multiplies<>`, `divides<>`, `modulus<>`
- Binary bitwise: `bit_and<>`, `bit_or<>`, `bit_xor<>`
- Comparison: `equal_to<>`, `not_equal_to<>`, `less<>`, `less_equal<>`, `greater<>`, `greater_equal<>`

[*Note:* Shift operators are not listed because the C++ standard library does not currently provide transparent function objects for them. See [P4006]. — *end note*]

[*Note:* When element-wise application is used, scalar operators are invoked with elements as prvalues. If a user-defined type provides both lvalue-reference-qualified and rvalue-reference-qualified overloads of an operator, the rvalue-reference-qualified overload is selected. This is an observable property of the element-wise application. — *end note*]

§ 9.6.1.2. Conversion customization [simd.cust.convert]

For conversions between `basic_vec` types where at least one of the source or destination element types is a user-defined vectorizable type, the implementation determines the result as follows:

1. If the expression `simd_convert(source, convert_to<To>)` is well-formed via ADL, where `source` is of type `const basic_vec<From, Abi>&` and the return type is exactly `basic_vec<To, Abi>`, then the result is that expression.

2. Otherwise, the result is element-wise `static_cast<To>(source[i])` for each i in the range `[0, basic_vec<From, Abi>::size())`.

For conversions where both the source and destination element types are built-in vectorizable types, the `simd_convert` customization point is never consulted. The implementation uses its own optimized conversion.

[Note: The well-formedness check above is SFINAE-friendly. If ADL finds a `simd_convert` overload with an incorrect return type, it is not considered and the implementation falls through to element-wise `static_cast`. —end note]

[Note: The `convert_to_t<To>` tag argument (`[simd.convert.tag]`) enables ADL discovery of the customization point. Without the tag, the destination type `To` would appear only as a template parameter and would not contribute to ADL. —end note]

9.7. Modify `[simd.ctor]` broadcasting constructor

Modify the constraints for the broadcasting constructor `explicit constexpr basic_vec(value_type x)` in `[simd.ctor]`:

Constraints:

- ~~From is not an arithmetic type and does not satisfy `constexpr-wrapper-like`, or~~
- From is not an arithmetic type and does not satisfy `constexpr-wrapper-like` and `is_convertible_v<From, value_type>` is true, or
- From is an arithmetic type and the conversion from From to `value_type` is value-preserving (`[simd.general]`), or
- From satisfies `constexpr-wrapper-like`, `remove_cvref_t<decltype(From::value)>` is an arithmetic type, and `From::value` is representable by `value_type`.

Drafting note: This ensures that conversions involving user-defined types respect the type author's design. If the scalar type requires explicit conversion (e.g., `explicit Meters(float)`), the `simd` conversion also requires explicit construction. If the scalar type allows implicit conversion, `simd` follows suit.

9.8. Modify `[simd.ctor]` converting constructor

Modify the Remarks and Effects for the converting constructor `template<class U, class UAbi> explicit(see below) basic_vec(const basic_vec<U, UAbi>& x)` in `[simd.ctor]`:

Remarks: The expression inside `explicit` evaluates to `true` if any of the following hold:

- ~~the conversion from `U` to `value_type` is not value-preserving, or~~
- both `U` and `value_type` are built-in vectorizable types and the conversion from `U` to `value_type` is not value-preserving, or
- at least one of `U` or `value_type` is not a built-in vectorizable type and `is_convertible_v<U, value_type>` is false, or

The remaining conditions about integer conversion rank and floating-point conversion rank remain unchanged.

Effects: ~~Initializes the i^{th} element with `static_cast<value_type>(x[i])` for all i in the range `[0, size())`. If~~ both `U` and `value_type` are built-in vectorizable types, initializes the i^{th} element with `static_cast<value_type>(x[i])` for all i in the range `[0, size())` using an implementation-defined optimized conversion. Otherwise, determines the result according to the conversion customization rules (`[simd.cust.convert]`).

Drafting note: This extends the explicit-ness determination to user-defined vectorizable types. For UDT conversions, we check `is_convertible_v` rather than value-preserving (which is only defined for built-in vectorizable types).

The phrase "at least one of `U` or `value_type` is not a built-in vectorizable type" covers three cases:

1. **Built-in vectorizable** → **User-defined vectorizable** (e.g., `simd<float>` → `simd<Meters>`): requires and respects whether the UDT provides an implicit or explicit constructor from the built-in vectorizable type (e.g., `Meters(float)`)
2. **User-defined vectorizable** → **Built-in vectorizable** (e.g., `simd<Meters>` → `simd<float>`): requires and respects whether the UDT provides an implicit or explicit conversion operator to the built-in vectorizable type (e.g., `operator float()`)
3. **User-defined vectorizable** → **User-defined vectorizable** (e.g., `simd<Meters>` → `simd<Feet>`): requires and respects whether conversion is available and whether it's implicit or explicit. The UDT author can provide either a constructor in the target type (`Feet(Meters)`) or a conversion operator in the source type (`Meters::operator Feet()`) allowing `static_cast` to use whichever is available.

The Built-in vectorizable → Built-in vectorizable case is handled by the first condition's value-preserving check. We use "at least one" (not "both") because we want to respect the type author's implicit/explicit judgment for *any* conversion involving a UDT. The `is_convertible_v` check will fail (requiring explicit construction) if the necessary constructor or conversion operator doesn't exist or is marked explicit.

9.9. Modify [simd.unary]

Modify the constraints and effects in [simd.unary] as follows:

Let *op* be the operator.

Constraints: `requires (value_type a) { op a; } is true` `supported-unary-op<value_type, Op> is true`, where *Op* is the corresponding standard transparent function object (`negate<>`, `bit_not<>`).

Returns: A `basic_vec` object initialized with the results of applying *op* to *v* as a unary element-wise operation. If *value_type* is a built-in vectorizable type, a `basic_vec` object initialized with the results of applying *op* to *v* using an implementation-defined optimized implementation. Otherwise, a `basic_vec` object determined according to the operation customization rules ([simd.cust.op]).

9.10. Modify [simd.binary]

Modify the constraints and effects in [simd.binary] as follows:

Let *op* be the operator.

Constraints: `requires (value_type a, value_type b) { a op b; } is true` `supported-binary-op<value_type, Op> is true`, where *Op* is the corresponding standard transparent function object (`plus<>`, `minus<>`, `multiplies<>`, `divides<>`, `modulus<>`, `bit_and<>`, `bit_or<>`, `bit_xor<>`).

Returns: A `basic_vec` object initialized with the results of applying *op* to *lhs* and *rhs* as a binary element-wise operation. If *value_type* is a built-in vectorizable type, a `basic_vec` object initialized with the results of applying *op* to *lhs* and *rhs* using an implementation-defined optimized implementation. Otherwise, a `basic_vec` object determined according to the operation customization rules ([simd.cust.op]).

For the shift operators:

Let *op* be the operator.

Constraints: `requires (value_type a, simd_size_type b) { a op b; } is true` `supported-binary-op<value_type, Op> is true`, where *Op* is the corresponding standard transparent function object.

Returns: A `basic_vec` object initialized with the results of applying *op* to *lhs* and *rhs* as a binary element-wise operation. If *value_type* is a built-in vectorizable type, a `basic_vec` object initialized with the results of applying *op* to *lhs* and *rhs* using an implementation-defined optimized implementation. Otherwise, a `basic_vec` object initialized with the results of applying *op* element-wise.

[*Note*: Shift operators do not consult `simd_operator` because the C++ standard library does not provide transparent function objects for them. See [P4006]. If [P4006] is adopted, shift operators could be added to the customization mechanism. — *end note*]

9.11. Modify [simd.cassign]

Modify the constraints and effects in [simd.cassign] as follows:

Let *op* be the operator.

Constraints: ~~requires (value_type a, value_type b) { a op b; } is true~~ *supported-binary-op*<value_type, Op> is true, where Op is the standard transparent function object corresponding to the binary operator with the same name (e.g., `operator+=` uses the constraint from `operator+`) .

Effects: Equivalent to `lhs = lhs op rhs`, where the binary operation is performed according to the rules in [simd.binary].

For the shift compound assignment operators:

Let *op* be the operator.

Constraints: ~~requires (value_type a, simd_size_type b) { a op b; } is true~~ *supported-binary-op*<value_type, Op> is true, where Op is the standard transparent function object corresponding to the binary operator .

Effects: Equivalent to `lhs = lhs op rhs`, where the binary operation is performed according to the rules in [simd.binary].

9.12. Modify [simd.comparison]

Modify the constraints and effects in [simd.comparison] as follows:

Let *op* be the operator.

Constraints: ~~requires (value_type a, value_type b) { a op b; } is true~~ *requires* (value_type a, value_type b) { { a op b } -> same_as<bool>; } is true .

Returns: ~~A mask_type object initialized with the results of applying op to lhs and rhs as a binary element wise operation.~~ If value_type is a built-in vectorizable type, a mask_type object initialized with the results of applying *op* to lhs and rhs using an implementation-defined optimized implementation. Otherwise, a mask_type object determined according to the comparison customization rules ([simd.cust.op]).

9.13. Add overload for to_underlying

Add to [simd.casts]:

```
template<simd-vec-type V>
constexpr rebind_t<underlying_type_t<typename V::value_type>, V>
to_underlying(const V& v) noexcept;
```

Constraints: `is_enum_v<typename V::value_type>` is true.

Returns: A `basic_vec` object where the i^{th} element is initialized to the result of `to_underlying(v[i])` for all i in the range `[0, V::size())`.

§ 9.14. Add overload for `to_integer`

Add to `[simd.casts]`:

```
template<class IntegerType, class Abi>
constexpr rebind_t<IntegerType, basic_vec<byte, Abi>>
    to_integer(const basic_vec<byte, Abi>& v) noexcept;

Constraints: is_integral_v<IntegerType> is true.

Returns: A basic_vec object where the  $i^{\text{th}}$  element is initialized to the result of to_integer<IntegerType>(v[i])
for all  $i$  in the range [0, basic_vec<byte, Abi>::size()).
```

§ 9.15. Feature test macro `[version.syn]`

Add to `[version.syn]`:

```
#define __cpp_lib_simd_ugt YYYYMMML // also in <simd>
```

[*Note:* This macro covers both the trait-based vectorizable type extension and the customization point mechanism (`simd_operator` and `simd_convert`), as they form a single integrated feature. — *end note*]

§ 10. Conclusion

This proposal extends `std::simd` to support user-defined element types through a minimal, principled change where the closed list of vectorizable types is extended, with `std::byte` added directly, and trait-based constraints introduced for user-defined vectorizable types.

Earlier revisions explored explicit customization mechanisms, leading to complicated designs. Committee feedback encouraged exploring element-wise inference. The working draft specification already defines all operations through element-wise application, so changing only the definition of which types are allowed provides the extension we need.

Committee discussion raised legitimate concerns about whether compilers could actually optimize user-defined operator calls into efficient vector code. Implementation experience with leading compilers (Clang 20, Intel oneAPI 2025.0) has shown that they can. While compiler maturity varies across vendors and versions, the results demonstrate the fundamental viability of the element-wise inference approach.

By changing only the gate-keeping logic for vectorizable types, we enable type safety for strong typedefs, domain-specific types for signal processing and other specialized domains, enumerations, `std::byte`, and small compound types. This is achieved with no breaking changes to existing code and no modification to any operation semantics.

The proposal includes ADL-based customization points (`simd_operator` for operations, `simd_convert` for conversions) that enable users to provide optimized implementations where compiler inference is insufficient. The hybrid approach — element-wise inference by default, customization when needed — provides a clear path for users to achieve optimal performance regardless of compiler optimization quality.

§ 11. Acknowledgements

We would like to thank Matthias Kretz for his feedback and contributions to discussions throughout the development of this proposal. We also thank the members of SG1 and SG6 who provided feedback during recent meetings, which significantly shaped the direction of the later revisions.

§ 12. Appendix: Customization Point Technical Details

This appendix provides additional technical details for the ADL-based customization mechanism proposed in §6 [Customization Points](#).

§ 12.1. Dual Dispatch Strategy

The customization design uses separate code paths based on type category:

```
// Built-in vectorizable types : always optimized
template<typename T>
    requires std::is_arithmetic_v<T> || std::is_same_v<T, std::byte> || /* complex */
friend constexpr basic_vec operator+(const basic_vec& lhs, const basic_vec& rhs)
{
    return /* implementation-defined optimized implementation */;
}

// User-defined vectorizable types: check for customization via ADL
template<typename T>
    requires (!std::is_arithmetic_v<T> && !std::is_same_v<T, std::byte> && /* not complex */
friend constexpr basic_vec operator+(const basic_vec& lhs, const basic_vec& rhs)
    requires requires (value_type a, value_type b) { { a + b } -> std::same_as<value_type>;
{
    if constexpr (requires { simd_operator(lhs, rhs, std::plus<>{}); }) {
        return simd_operator(lhs, rhs, std::plus<>{}); // Custom via ADL
    } else {
        return /* element-wise application */; // Default
    }
}
}
```

This ensures:

- **Built-in vectorizable types:** Always optimized, never check for customization
- **User-defined vectorizable types (including enumerations) :** Can provide `simd_operator` customization, otherwise element-wise inference is used. For enumerations without custom operators, element-wise inference produces optimal code since enum operations compile to simple integer arithmetic.
- **Performance guarantee:** No overhead for standard built-in vectorizable types

Users control their own target-specific optimizations if desired:

```
// User code for target-specific optimization
namespace my_lib {
    enum class PackedColor : uint32_t { /* ... */ };

    // Custom enum operator
    PackedColor operator+(PackedColor a, PackedColor b) {
        return /* custom blending logic */;
    }

    // SIMD optimization, discoverable by ADL.
    auto simd_operator(vec<PackedColor> lhs, vec<PackedColor> rhs, std::plus<>) {
        #ifdef __AVX512F__
            return my_avx512_blend(lhs, rhs);
        #else
            return my_generic_blend(lhs, rhs);
        #endif
    }
}
```

12.2. Complete Example with Selective Customization

This example shows how users can customize specific operations while relying on element-wise inference for others:

```
namespace my_lib {
    struct fixed_point_16s8 {
        std::int16_t data;

        // Basic operators use normal semantics
        fixed_point_16s8 operator+(fixed_point_16s8 rhs) const {
            return fixed_point_16s8{data + rhs.data};
        }

        fixed_point_16s8 operator-(fixed_point_16s8 rhs) const {
            return fixed_point_16s8{data - rhs.data};
        }

        bool operator<(fixed_point_16s8 rhs) const {
            return data < rhs.data;
        }
    };

    // Customize multiply (requires scaling) - Binary operation
    template<typename Abi>
    auto simd_operator(
        const basic_vec<fixed_point_16s8, Abi>& lhs,
        const basic_vec<fixed_point_16s8, Abi>& rhs,
        std::multiplies<>)
    {
        // Custom implementation with appropriate scaling
        // Could use intrinsics or library functions
        return /* optimized multiply with scaling */;
    }

    // Customize divide (requires scaling) - Binary operation
    template<typename Abi>
    auto simd_operator(
        const basic_vec<fixed_point_16s8, Abi>& lhs,
        const basic_vec<fixed_point_16s8, Abi>& rhs,
        std::divides<>)
    {
        // Custom implementation with appropriate scaling
        return /* optimized divide with scaling */;
    }

    // Addition, subtraction, comparisons use element-wise inference
    // No customization needed for these simple operations
}

// Usage
vec<my_lib::fixed_point_16s8> a, b;
auto sum = a + b;      // Uses element-wise inference (fast)
auto diff = a - b;     // Uses element-wise inference (fast)
auto product = a * b;  // Uses custom simd_operator (optimal)
auto quotient = a / b; // Uses custom simd_operator (optimal)
auto mask = a < b;     // Uses element-wise inference (fast)
```

Conversion example:

```
namespace my_lib {
    struct BFloat16 { uint16_t bits; /* ... */ };

    // Optimize conversion to float
    template<typename Abi>
```

```

basic_vec<float, Abi>
simd_convert(const basic_vec<BFloat16, Abi>& source, convert_to_t<float>) {
    // Use hardware bfloat16 conversion if available
    #ifdef __AVX512BF16__
        return /* use vcvtneps2bf16 or similar */;
    #else
        return /* shift bits implementation */;
    #endif
}
}

```

This demonstrates the key benefit: users customize only what needs optimization while relying on inference for everything else. The single `simd_operator` name handles unary, binary, and comparison operations through overloading.

§ 13. Appendix: Assembly Code Examples

This section provides detailed assembly listings from the implementation experience, demonstrating how element-wise inference generates optimal vector code. Testing was performed with Clang 20 and Intel oneAPI 2025.0 targeting Intel Sapphire Rapids.

Complex Expression Composition

Element-wise operations compose well across multiple operations in a single expression:

C++ Code	Generated Assembly
<pre> // Strong typedef auto compute(vec<Meters> a, vec<Meters> b, vec<Meters> c) { return (a + b) * c - a; } // Built-in type (for comparison) auto compute(vec<float> a, vec<float> b, vec<float> c) { return (a + b) * c - a; } </pre>	<pre> ; Strong typedef Meters compute(...): vaddps zmm1, zmm1, zmm0 vfmsub231ps zmm0, zmm2, zmm1 ret ; Built-in float compute(...): vaddps zmm1, zmm1, zmm0 vfmsub231ps zmm0, zmm2, zmm1 ret </pre>

The assembly is identical for both the user-defined type and the built-in type, demonstrating that user-defined types achieve zero-overhead abstraction. The compiler successfully fuses multiple operations and optimizes register allocation regardless of whether the element type is `Meters` or `float`.

C++ Code	Generated Assembly
<pre> auto broadcast(int16_t x) { return vec<saturating_int16>(x); } </pre>	<pre> broadcast(short): vpbroadcastw zmm0, edi ret </pre>
<pre> auto iq_swap(const vec<saturating_int16>& v) { return permute(v, [](auto idx) { return idx ^ 1; }); } </pre>	<pre> iq_swap(...): vprold zmm0, zmmword ptr [rdi], 1 ret </pre>
<pre> auto add(vec<saturating_int16> lhs, vec<saturating_int16> rhs) { </pre>	<pre> add(...): vpaddsw ymm0, ymm0, ymm1 ret </pre>

C++ Code	Generated Assembly
<pre> return lhs + rhs; } </pre>	
<pre> auto compound_add(vec<saturating_int16> lhs, vec<saturating_int16> rhs) { lhs += rhs; return lhs; } </pre>	<pre> compound_add(...): vpaddsw ymm0, ymm0, ymm1 ret </pre>
<pre> auto cmp_gt(vec<saturating_int16> lhs, vec<saturating_int16> rhs) { return lhs > rhs; } </pre>	<pre> cmp_gt(...): vpcmpgtw ymm0, ymm1, ymm0 ret </pre>
<pre> auto biggest(vec<saturating_int16> lhs, vec<saturating_int16> rhs) { return max(lhs, rhs); } </pre>	<pre> biggest(...): vpmasw ymm0, ymm0, ymm1 ret </pre>
<pre> auto distance(vec<Meters, 8> x, vec<Meters, 8> y) { return x + y; } </pre>	<pre> distance(...) vaddps ymm0, ymm0, ymm1 ret </pre>
<pre> auto closer(vec<Meters> x, vec<Meters> y) { return x < y; } </pre>	<pre> closer(...) vcmpltb ymm0, ymm0, ymm1 ret </pre>
<pre> auto dimmer(vec<Color> x, vec<Color> y) { return x < y; } </pre>	<pre> dimmer(...) vpcmpgtb ymm0, ymm1, ymm1 ret </pre>
<pre> auto load_and_convert(std::span<short, 1024> s) { return unchecked_load<vec<Meters, 8>>(s); } </pre>	<pre> load_and_convert(...): # vcvtqdq2ps ymm0, ymmword ret </pre>
<pre> auto gather(std::span<int, 1024> s, const vec<int, 8> indexes) { return unchecked_gather_from<vec<Meters, 8>>(s, indexes); } </pre>	<pre> gather(): # kxnorb k1, k0, k0 vpxor xmm1, xmm1, xmm1 vpgatherdd ymm1 {k1}, ymm0 vcvtqdq2ps ymm0, ymm1 ret </pre>

These examples demonstrate optimal code generation with native vector instructions and no scalar fallback.

References

Informative References

[P0122R7]

Neil MacIntosh; Stephan T. Lavavej. *span: bounds-safe views for sequences of objects*. URL: <https://wg21.link/P0122R7>

[P3045R4]

Mateusz Pusz; et al. *Quantities and units library*. URL: <https://wg21.link/P3045R4>

[P3081R0]

Jarrad Waterloo. *Core safety profiles: Specification, adoptability, and impact*. URL: <https://wg21.link/P3081R0>

[P4006]

Daniel Towner. *Transparent wrappers for shift operators*. URL: <https://wg21.link/P4006>

[SIMD.GENERAL]

General requirements for SIMD types. URL: <https://eel.is/c++draft/simd#general>